

Accredited BY NBA | AICTE Sponsored Margdarshan Mentor Institution

DST-FIST Supported Institution | ISO 9001:2015 Certified

Recognized under Section 2(f) & 12(B) of the UGC Act, 1950



A PROJECT REPORT

Submitted by

CEVANA S M (95072515022)

ASWINI DEVI R (95072515019)

ARUNA SRI G (95072515018)

In partial fulfillment for the award of the

degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

FRANCIS XAVIER ENGINEERING COLLEGE

(Autonomous)

TIRUNELVELI-627003

MAY-2026

FRANCIS XAVIER ENGINEERING COLLEGE

(Autonomous)

TIRUNELVELI-627003

BONAFIDE CERTIFICATE

Certified that this project report “***PASSWORD GENERATOR AND STRENGTH CHECKER***” is the bonafide work of “***ASWINI DEVI R (95072515019)***” who carried out the project work under my supervision. Certified further that to the best of my knowledge, the work reported here in does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SUPERVISOR

HEAD OF THE DEPARTMENT

Submitted for the B. TECH Degree Project Viva Voce held on.....

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

<i>S.NO</i>	<i>TITLE</i>	<i>PAGE NO.</i>
<i>1</i>	ABSTRACT	1
<i>2</i>	OBJECTIVE	2
<i>3</i>	ALGORITHM STRATEGY	3
<i>4</i>	FLOWCHART	4
<i>5</i>	SOURCE CODE	5
<i>6</i>	OUTPUT	6
<i>7</i>	FEATURES	8
<i>8</i>	FUTURE ENHANCEMENTS	8
<i>9</i>	CONCLUSION	9

ABSTRACT

The Password Generator and Strength Checker is a Python-based console application developed to enhance password security and create awareness about strong password practices. In today's digital era, passwords are used everywhere such as social media accounts, banking applications, email services, and online shopping platforms. Since most people use simple and predictable passwords, they become easy targets for hackers. This project is created to help users generate strong passwords and check the strength of passwords they already use. Passwords act as the first line of defense for protecting personal data and confidential information. Weak passwords can lead to serious issues such as account hacking, identity theft, and loss of important files. Many users reuse the same password for multiple accounts, which increases the risk of cyberattacks. This application solves that problem by providing unique and secure password generation features that reduce the chance of unauthorized access. The password generator part of this project creates random passwords based on the user's requirements. The user can select the password length and decide whether the password should include uppercase letters, lowercase letters, numbers, and special characters. The program uses Python libraries such as random and string to generate unpredictable and secure passwords. This ensures that every password generated is different and difficult to guess. Another important feature of this project is the password strength checker. Many users are unaware of whether their password is strong or weak. The strength checker analyzes a given password and evaluates its security based on multiple factors. It checks conditions like password length, presence of digits, use of special characters, and combination of uppercase and lowercase letters. Based on these conditions, it gives the result as weak, medium, or strong.

OBJECTIVE

The objective of the program is to develop a Password Generator and Strength Checker using Python that allows users to generate strong and secure passwords for their online accounts. The program is designed to provide a simple and efficient way to create passwords that are difficult to guess, ensuring better safety for personal and confidential information.

Another important objective of this project is to help users understand the importance of strong passwords in today's digital world. Many users create weak passwords such as names, mobile numbers, or simple patterns which can be easily cracked. This program aims to reduce such risks by encouraging users to use complex and random passwords.

The password generator part of the project aims to produce passwords using a combination of uppercase letters, lowercase letters, numbers, and special characters. The user can choose the password length and select the type of characters to be included. This objective ensures that the generated password is unique and meets security requirements. The strength checker part of the program aims to analyze any password entered by the user and determine whether it is strong enough. The program checks different conditions such as password length, presence of digits, special symbols, and mixture of upper and lower case letters. Based on these checks, it gives the result as weak, medium, or strong.

Overall, the objective of this project is not only to create a working Password Generator and Strength Checker but also to provide learners with a strong foundation in Python programming and cybersecurity awareness. It improves logical thinking, enhances problem-solving skills, and prepares students to build more advanced security-based applications such as password managers, encrypted storage systems, and GUI-based password tools in the future.

SIMPLE CALCULATOR ALGORITHM STRATEGY

Calculator Concept:

Represents a basic calculator application.
Performs arithmetic operations.
Uses user input to process calculations and display results.

Input Handling:

Gets user choice (Generate / Check / Exit).
Takes password length and password input.
Converts input properly to avoid errors.

Password generation algorithm:

User enters required password length.
User selects character types (A-Z, a-z, 0-9, symbols).
Program generates and displays random password.

Random Character Selection:

Uses `random.choice()` to select characters.
Repeats selection until password length is completed.
Ensures password is random and unpredictable.

Password Strength Checking Algorithm:

Takes password from user for checking.
Checks length, uppercase, lowercase, digit, and symbol.
Gives result as Weak, Medium, or Strong.

Weak password condition:

Password length is too small.
Missing digits or special characters.
Strength is marked as Weak.

Strong password condition:

Password length is long enough.
Contains uppercase, lowercase, digits, and symbols.
Strength is marked as Strong.

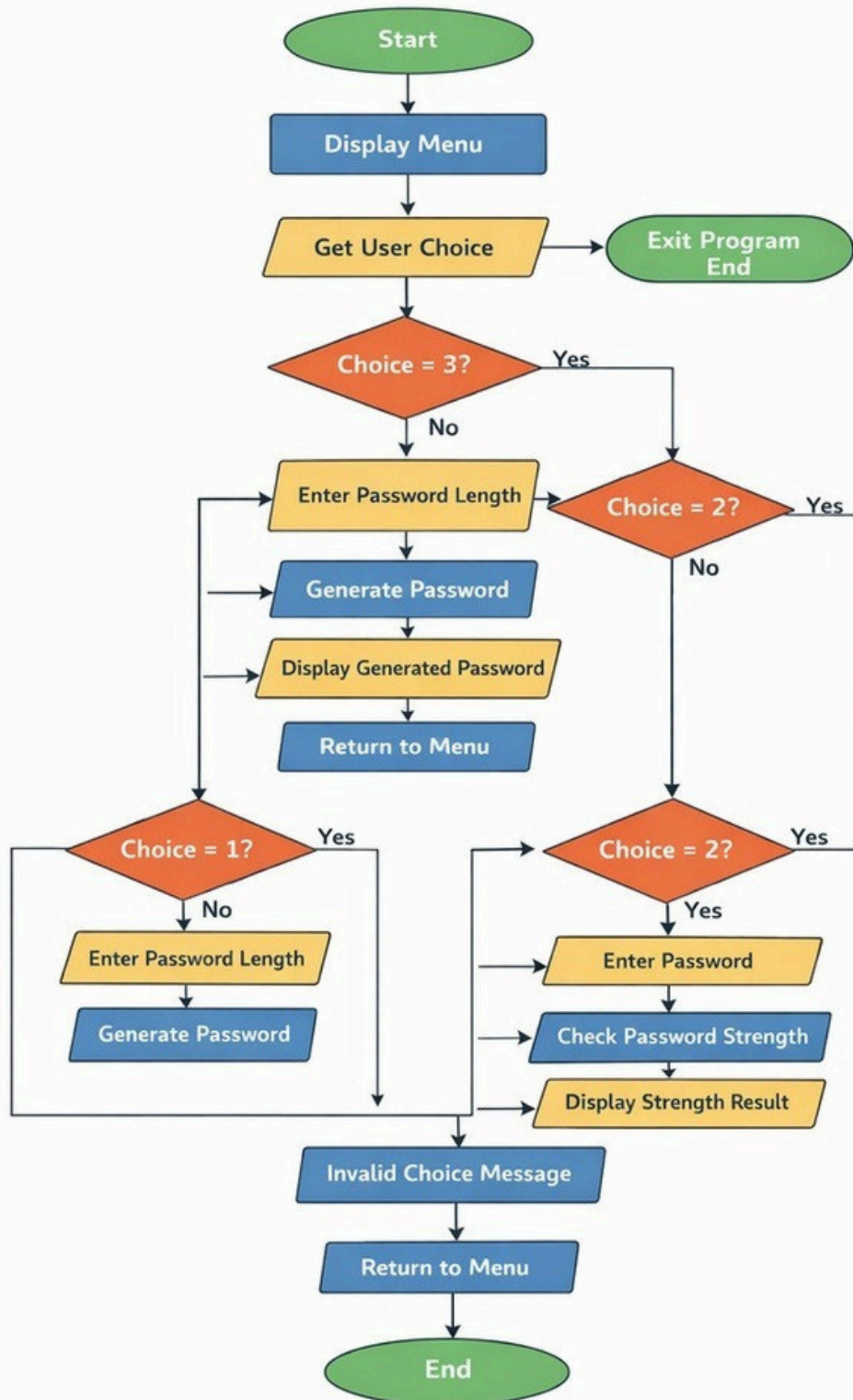
Menu Control Algorithm:

Password length is long enough.
Contains uppercase, lowercase, digits, and symbols.
Strength is marked as Strong.

Result Display Algorithm:

Displays generated password clearly.
Displays password strength result.
Suggests improvements for weak passwords.

FLOW CHART



SOURCE CODE

```
import random
import string
# Function to generate password
def generate_password(length):

    characters = string.ascii_letters + string.digits + string.punctuation
    password = ""
    for i in range(length):

        password += random.choice(characters)

    return password

#Function to check password strength
def check_strength(password):
    length = len(password)
    has_upper = False
    has_lower = False
    has_digit = False
    has_symbol = False
    for ch in password:

        if ch.isupper():
            has_upper = True
        elif ch.islower():
            has_lower = True
        elif ch.isdigit():
            has_digit = True
        elif ch in string.punctuation:
            has_symbol = True

    score = 0
    if length >= 8:

        score += 1
        if has_upper:
            score += 1
        if has_lower:
            score += 1
        if has_digit:
            score += 1
        if has_symbol:
            score += 1
```

```
if score <= 2:
    return "Weak Password"
elif score == 3 or score == 4:
    return "Medium Password"
else:
    return "Strong Password"
```

```
#Main Program
```

```
while True:
    print("\n--- Password Generator and Strength Checker ---")
    print("1. Generate Password")
    print("2. Check Password Strength")
    print("3. Exit")
    choice = input("Enter your choice (1-3): ")
    if choice == '3':
```

```
        print("Exiting Program...")
        break
```

```
elif choice == '1':
    try:
        length = int(input("Enter password length: "))
        if length < 4:
            print("Password length should be at least 4.")
        else:
            password = generate_password(length)
            print("Generated Password:", password)
    except ValueError:
        print("Invalid input! Please enter a number.")
```

```
elif choice == '2':
    password = input("Enter password to check strength: ")
    result = check_strength(password)
    print("Password Strength:", result)
```

```
else:
    print("Invalid choice! Please select from 1 to 3.")
```

OUTPUT

--- Password Generator and Strength Checker ---

1. Generate Password
2. Check Password Strength
3. Exit

Enter your choice (1-3): 1

Enter password length: 12

Generated Password: jA4#kL9*pM1!

--- Password Generator and Strength Checker ---

1. Generate Password
2. Check Password Strength
3. Exit

Enter your choice (1-3): 2

Enter password to check strength: qwerty

Password Strength: Weak Password

--- Password Generator and Strength Checker ---

1. Generate Password
2. Check Password Strength
3. Exit

Enter your choice (1-3): 1

Enter password length: 3

Password length should be at least 4.

--- Password Generator and Strength Checker ---

1. Generate Password
2. Check Password Strength
3. Exit

Enter your choice (1-3): 2

Enter password to check strength: P@rongPassword@1

Password Strength: Strong Password

--- Password Generator and Strength Checker ---

1. Generate Password
2. Check Password Strength
3. Exit

Enter your choice (1-3): 3

Exiting Program...

FEATURES

- ***RandomPasswordGeneration:***
Generates strong and unique passwords using random characters.
- ***UserInputHandling:***
Accepts user inputs such as password length and strength-check password.
- ***CustomCharacterSelection:***
Allows users to choose uppercase, lowercase, digits, and special symbols.
- ***SecureCharacterPoolCreation:***
Combines selected character types into a single pool for password generation.
- ***PasswordStrengthChecking:***
Checks password strength based on length, digits, uppercase, lowercase, and symbols.
- ***StrengthClassification:***
Classifies passwords as Weak, Medium, or Strong based on conditions satisfied.
- ***ErrorHandling:***
Handles invalid input and prevents errors such as empty character selection.
- ***Menu-DrivenInterface:***
Provides simple menu options to generate password, check strength, or exit.
- ***ClearResultDisplay:***
Displays generated password and strength result in a user-friendly format.

FUTURE ENHANCEMENT

- ***AdvancedPasswordRules:***
Add options like minimum symbol count, minimum digit count, and required uppercase letters.
- ***PasswordSuggestionFeature:***
Suggest improvements when a password is weak or medium.
- ***Password Encryption Storage:***
Store generated passwords securely using encryption techniques.
- ***Graphical UserInterface(GUI):***
Create a GUI version using Tkinter or PyQt for better user experience
- ***Password Copy Option:***
Add a feature to copy generated password directly to clipboard.
- ***Strength Score Display:***
Show password strength score in percentage or rating format.

CONCLUSION

The Password Generator and Strength Checker project in Python successfully demonstrates the importance of creating secure passwords and ensuring data protection in today's digital world. This project is very useful because passwords play a major role in protecting personal information, online accounts, and confidential data. By developing this project, users can understand how strong passwords can prevent hacking and unauthorized access. This project mainly focuses on generating random passwords using a combination of letters, numbers, and special characters. The password generator produces unique and unpredictable passwords, which are much safer than simple passwords like names or birthdates. This makes the program highly valuable in real-life situations where strong password creation is necessary. The project also includes a password strength checker, which helps users test whether a password is weak, medium, or strong. This strength checking system improves user awareness by showing how password quality depends on factors such as length and character variety. It teaches users that a password must be complex to be considered secure. This project also teaches the importance of handling errors such as invalid user input or incorrect password length. By adding basic validation, the program becomes more reliable and user-friendly. It helps learners understand how proper error handling improves the overall performance of a program. Overall, the Password Generator and Strength Checker project is an excellent learning tool for beginners. It improves logical thinking and problem-solving skills by combining multiple Python concepts into one practical application. Finally, this project serves as a strong foundation for future development and advanced learning. It can be extended into a more powerful system such as a GUI-based password generator, password saving system, encrypted password manager, or login authentication program. Therefore, this project is not only useful for learning Python but also acts as a valuable step towards developing real-world cybersecurity applications.